K-Waay 接收端安全问题的形式化分析与组合加固

作者信息待补

（作者单位、城市、邮编待补）

摘 要： 针对 K-Waay 会话参数失配与批内重复接受问题，利用 ProVerif 和 Tamarin 分析核心交换与 BatchReceive。发现攻击者复用合法 split-KEM 密文、重构其他 KEM 密文，可使接收方在会话密钥未泄露时接受与发送方密钥不一致的会话。HMAC 密钥确认使无泄露模型中的完整参数对应关系由不成立转为成立。在未维持参与方互异条件的两槽模型中，合法确认消息仍可被重复接受；独立消费模型进一步验证了条件化重复符号安装。为此，提出处理前的批内原子消息去重，并与 HMAC 确认组合。验证表明，批内重复接受及条件化重复安装被阻断，批内接收单射性成立，正常批次处理与输出消费仍可达。

关键词： K-Waay；密文重组攻击；批内重放；HMAC 密钥确认；原子去重；ProVerif；Tamarin

中图分类号：待确认 文献标志码：待确认 文章编号：待定

Formal Analysis and Combined Hardening of Receiver-Side Security in K-Waay

Author information pending

(Affiliation, city, postal code, and country pending)

Abstract: Receiver-side session mismatch and intra-batch duplicate acceptance in K-Waay were analyzed using ProVerif and Tamarin. The analysis identified a ciphertext recomposition attack: an adversary could reuse a legitimate split-KEM ciphertext and reconstruct other KEM ciphertexts, causing the receiver to accept a session key different from the sender's without learning that key. HMAC-based key confirmation restored full-parameter correspondence in the uncompromised symbolic model. However, when the distinct-party precondition was not maintained, a valid confirmed message could still be accepted twice within a two-slot batch. A separate output-consumption model further demonstrated conditional duplicate symbolic installations. An atomic, batch-local message deduplication mechanism was therefore proposed and combined with HMAC confirmation. Verification showed that duplicate acceptance and conditional duplicate installation were prevented, while batch-scoped acceptance injectivity and normal batch processing and output consumption remained achievable.

Key words: K-Waay; ciphertext recomposition attack; intra-batch replay; HMAC-based key confirmation; atomic deduplication; ProVerif; Tamarin

0 引言

随着后量子密码技术的发展，安全消息协议逐渐采用基于密钥封装机制（Key Encapsulation Mechanism，KEM）的密钥交换方案，以应对量子计算对传统公钥密码体系的潜在威胁。K-Waay 是一种面向异步通信的后量子可否认认证密钥交换协议，通过 KEM、数字签名与 split-KEM 实现类似 X3DH 的密钥交换功能[2-3]。该协议支持发送方离线发起会话，并通过 BatchReceive 机制处理多个发送方使用同一接收方临时状态生成的消息。然而，接收方能否正确确认完整会话参数，以及批量处理过程中是否存在重复接受行为，仍需要结合具体接收流程进行分析。

已有研究对安全消息协议的认证与密钥安全性进行了深入分析。例如，Signal 协议的形式化研究系统考察了密钥交换及后续消息处理中的安全性质[4]，PQXDH 的形式化验证进一步研究了后量子密钥协商的认证和机密性[5]。这些研究为分析后量子消息协议提供了方法基础，但密钥机密性、发送与接收会话参数一致性以及批次内部的接收唯一性是不同的安全目标。尤其对于采用多个密码分量构造会话密钥的协议，单个分量的来源合法性并不必然意味着接收方接受的完整消息与发送方实际生成的消息一致；对于批量接收机制，单条消息的有效性验证也不能直接保证该消息不会被重复处理。

针对上述问题，本文首先利用 ProVerif 对 K-Waay 核心密钥交换过程进行形式化分析，发现一条密文重组攻击路径。攻击者能够保留合法发送方产生的 split-KEM 密文，同时利用公开密钥重新构造其他 KEM 密文，使接收方在成功解封装后接受篡改消息，进而导出与发送方不同的会话标识及会话密钥。该攻击不需要获得接收方会话密钥，说明会话密钥机密性成立并不足以保证双方会话参数一致。为阻断该攻击，本文提出基于 HMAC 的密钥确认机制，使接收方在完成接收前验证由会话密钥及完整转录本标识生成的确认标签。

进一步地，本文使用 Tamarin 分析 K-Waay 的 BatchReceive 机制，发现在参与方互异前置条件未得到维持的情况下，即使消息已经通过 HMAC 确认，攻击者仍能够将同一合法消息重复投递至同一批次的不同槽位，造成一个发送会话对应多个接收事件。在逐输出独立消费的条件下，重复接收还可能进一步产生具有相同会话参数的多个本地安装事件。针对这一问题，本文提出处理前批内原子消息去重机制，并与 HMAC 确认相结合，在消息接受之前统一检查重复输入，从而实现会话参数确认与批内接收唯一性的分层保护。

本文主要贡献如下：

1. 发现 K-Waay 核心消息篡改攻击并提出修复方案。 通过 ProVerif 构造密文重组攻击，揭示接收方在会话密钥未泄露的情况下仍可能接受失配会话的问题；提出 HMAC 密钥确认机制，并验证其能够恢复无泄露模型中的完整会话参数对应关系。

2. 发现 HMAC 确认后的批内重复接受问题并分析安全影响。 利用 Tamarin 构造合法确认消息的批内重放路径，证明同一发送消息能够对应两个不同接收事件，并

基金项目：待补

作者简介：待补

通信作者及电子邮箱：待补

在独立的条件化消费模型中验证重复接收可能造成的重复安装后果。

3. 提出批内原子去重与 HMAC 组合加固方案。 在批次处理前增加完整消息去重检查，阻断重复输入产生多个成功接收事件；进一步结合 HMAC 确认，通过形式化验证表明组合方案能够阻断相应攻击路径，同时保持正常密钥交换与批次处理能力。

1 K-Waay 协议及安全问题

1.1 K-Waay 核心密钥交换流程

K-Waay 是面向异步通信的后量子可否认认证密钥交换协议，利用 KEM、签名和 split-KEM 实现发送方离线发起与接收方后续处理[2]。其核心接口包括密钥生成、预密钥初始化、发送及批量接收。本文关注 Figure 7 所示核心构造及完整稿中的接收接口，而非对原协议计算安全证明的重新推导[3]。

设发送方为 A、接收方为 B。双方发布带签名的预密钥材料；发送方先验证接收方预密钥签名，再分别针对接收方长期 KEM 公钥和临时 KEM 公钥封装，并结合自身 split-KEM 秘密材料与接收方 split-KEM 公钥生成第三个分量。将三份密文及对应共享秘密记为 (ct_l, K_l)、(ct_k, K_k) 和 (ct_s, K_s)。核心消息、会话标识与会话密钥表示为

$$\begin{aligned} m &= (ct_l, ct_k, ct_s), \\ sid &= \mathit{SID}(A, B, pk_A, pk_B, \\ &\quad prekey_A, prekey_B, m), \\ k &= \mathit{KDF}(K_l, K_k, K_s, sid). \end{aligned} \tag{1}$$

其中，SID 是对原协议身份、公钥、预密钥和完整消息转录本的简记，并非新增的密码算法。发送方公开发送 m，会话密钥 k 不随消息公开。接收方先验证所声称发送方的预密钥签名，再使用本地长期、临时和 split-KEM 秘密材料解封装相应密文，并按相同字段顺序计算 sid 与 k；其接收方预密钥来自本地状态，而不是由网络消息替换。

这里必须区分三种保证：*解封装成功*表示局部密码处理可完成；*密钥机密性*要求攻击者不能推导会话密钥；*完整参数对应关系*则要求接收方的会话参数存在匹配的发送方完成过程。前两者不能直接推出第三者，这构成后文分析接收行为的起点。

1.2 BatchReceive 批量接收机制

异步场景下，多名发送方可能使用接收方已发布的同一临时预密钥。K-Waay 因而允许接收方针对共享的本地临时状态调用一次 BatchReceive，处理一个输入向量[3]：

$$S = \left((pk_{A_i}, prekey_{A_i}, m_i)\right)_{i=1}^{d}, d \geq 1. \tag{2}$$

每个位置关联一个发送方身份材料、预密钥束及消息；接收结果按照相应位置组织为密钥或失败值。原构造既有条目级失败处理，也有特定解封装失败导致的批次级失败，因此槽位中的局部计算不能无条件等同于最终批次成功返回。

完整稿第 5.1 节明确要求，一次 BatchReceive 调用的不同输入条目对应*不同参与方*[3]。记 A_i 为第 i 个条目的参与方投影，该前置条件为

$$\begin{aligned} &\mathit{DistinctPartyPerBatch}(S) \triangleq \\ &\forall i \neq j, A_i \neq A_j. \end{aligned} \tag{3}$$

该要求仅约束同一次调用内的条目，不禁止同一参与方在其他批次或其他会话中再次发送。它比较的是参与方，而非消息字节、预密钥或发送发生。本文后续考察这一既有输入条件*未被维持*时的放宽输入模型，不将所得反例称为满足原协议前置条件的执行。

1.3 接收端安全问题与研究范围

本文考察两个相互关联但并不等价的问题：其一，接收方完成会话时，是否存在身份、sid 和 k 均匹配的发送方完成事件；其二，即使一条消息具有匹配发送来源或通过显式密钥确认，在放宽批次输入条件后，它是否仍可在同一批次中被重复接受。前者属于*完整参数对应关系*，后者属于*批内接收发生唯一性*。重复接受可能产生何种上层效果，尚需额外说明接收输出的消费方式，不能从接收事件本身直接推出真实会话安装。

为分别回答上述问题，本文对核心密码处理建立 ProVerif 单次接收符号模型，对共享状态、槽位及发生顺序建立 Tamarin 有界批次模型，并在明确前提下比较加固变体。研究结论仅适用于各自的符号模型及假设，不替代 K-Waay 原有的计算安全结论。

2 形式化建模与安全性质

2.1 攻击者模型与研究假设

本文采用 Dolev--Yao 主动网络攻击者：攻击者可读取公开材料，拦截、重组、替换及重放消息，但只能使用符号模型允许的构造与消去规则。普通 KEM 封装可由公开公钥执行，因此攻击者能够构造新的 ct_l、ct_k；在没有相应 split-KEM 秘密材料时，不能凭空生成匹配的诚实 ct_s，但可以复用已观察到的分量。主要无泄露实验不开放秘密状态，独立泄露实验仅按各自模型解释。

批次模型还允许攻击者在抽象输入边界选择条目及其槽位，包括重复投递已观察到的有效消息。这是*模型中的批次组合能力*，并不证明攻击者能够操控真实部署的批次构造组件。下述 ProVerif 与 Tamarin 模型有不同的事件和密码学抽象，彼此结果不构成一条自动衔接的攻击轨迹。

2.2 ProVerif 核心模型

ProVerif 模型保留式(1)中的三密文消息、预密钥签名检查、KEM/split-KEM 解封装和密钥派生，以公开信道表示网络。发送方从网络获得声称的接收方预密钥；接收方使用本地保存的接收方预密钥与解封装秘密。模型采用*单次接收抽象*，不包含 BatchReceive 的批次槽位与关闭状态。

双方导出密钥时分别记录 $\mathit{SenderKey}(A, B, sid, k)$ 与 $\mathit{ReceiverKey}(B, A, sid, k)$；发送方完成和接收方完成分别记录

$$\begin{aligned} &\mathit{SendDone}(A, B, sid, k), \\ &\mathit{RecvDone}(B, A, sid, k). \end{aligned} \tag{4}$$

发送方的完成事件位于核心消息输出之后；原始模型的接收完成事件位于成功解封装及密钥派生之后。另以分量发送和分量接受事件检查 split-KEM 来源。上述事件是*模型观察点*，$\mathit{RecvDone}$ 不表示真实软件已安装会话。独立的 HMAC 变体在接收完成前增加确认检查；具体机制及验证对照留待第 3 章。

2.3 Tamarin 批次模型

Tamarin 使用多集重写规则表达线性状态和事件先后关系[6]。接收状态实例以 rst 标识，批次以 bid 标识，同批槽位以 idx 区分；输入收集、封闭、逐槽处理及批次完成或失败由线性事实约束。这些阶段是本文的*符号建模分解*，并非原协议新增加的算法接口。有界批次模型用于验证状态和生命周期约束；批内重放与后续加固采用固定两槽模型。

在历史重放模型中，发送完成事件为 $\mathit{SenderSession}(A, B, m, sid, k)$；接收发生事件为

$$\mathit{ReceiverAccept}(B, A, bid, idx, rst, m, sid, k). \tag{5}$$

模型以持久的诚实会话来源事实抽象成功解封装与参数匹配，同时保留不同槽位对同一来源的重复读取可能性；它*不独立重演* ProVerif 中的全部密码检查。HMAC-only 模型增加

确认消息及对应接受事件；影响模型另引入有条件的输出消费与符号安装层。这些理论的同名事件不能直接互换。

2.4 安全性质与验证分工

本文将安全目标分层，而不视为可互相替代的单一认证性质。P0-S 检查发送方或接收方密钥事件中的 k 是否可由攻击者推导；P0-O 检查已接受 split-KEM 分量是否具有匹配发送来源。P1 要求每个接收完成具有更早且参数完全一致的发送完成，形式为

$$RecvDone(B,A,sid,k)@r \Longrightarrow$$
$$\exists s < r: SendDone(A,B,sid,k)@s. \quad (6)$$

该性质为*非单射对应关系*，即同一个发送完成可以匹配多次接收；它不能保证重放防护。

P2 针对有界批次中的接收*发生*。令 $\theta = (A,B,m,sid,k)$、$\beta = (bid,rst)$，$R(\theta,\beta,idx)@r$ 表示式(5)中的接收事件，且发送来源的存在性与唯一性分别受相应引理约束。批内发生单射性的核心限制为

$$R(\theta,\beta,i)@r_1 \wedge R(\theta,\beta,j)@r_2$$
$$\Longrightarrow r_1 = r_2. \quad (7)$$

式(7)仅限制同一匹配消息及同一 (bid,rst) 内的重复接受，不推出不同消息所属参与方必然不同。P3 则在显式的 C_{install}-v2 消费假设下检查同一匹配发送是否引起重复符号安装；具体输出来源和安装唯一性定义留到第 4 章。参与方级准入另由式(3)刻画，不能由 P2 或精确消息去重直接替代。

表 1 安全性质与验证模型的对应关系

性质	检查目标	工具	适用边界
P0-S	会话密钥不可推导	ProVerif	无泄露符号基线
P0-O	split-KEM 分量有来源	ProVerif	分量级对应
P1	接收完成存在匹配发送	ProVerif	完整参数、非单射
P2	同批相同消息至多接受一次	Tamarin	固定两槽、同状态
P3	同源输出的安装唯一性	Tamarin	条件化消费模型

表 1 概括两类工具的分工。ProVerif 负责单次核心过程的机密性及对应关系；Tamarin 负责有界状态化批次中的重复接受、条件化影响。存在性见证与全称安全性质分别报告；后续各章仅在对应模型的范围内解释证明和反例，不将不同理论的事件拼接为未经验证的联合执行。

3 K-Waay 消息篡改攻击与 HMAC 确认加固

3.1 消息篡改攻击的发现

针对 K-Waay 核心密钥交换过程中发送方与接收方的会话一致性问题，本文利用 ProVerif 对原始协议的消息处理和密钥派生过程进行形式化分析，发现了一条能够诱导接收方接受篡改消息并产生会话密钥失配的攻击路径。

在该攻击中，攻击者无需获取发送方或接收方的长期秘密密钥，也无需破解 KEM 或 split-KEM 算法，而是利用公开密钥重新构造协议消息中的部分密文分量。接收方收到篡改后的消息后，仍能够完成预密钥验证、密文解封装和会话密钥派生，并在原始接收流程中触发接收完成事件。然而，由于接收方所处理的完整消息已经发生变化，其导出的会话标识及会话密钥与原发送方不一致，造成双方会话密钥失配。

本文首先验证了原始核心模型中的会话密钥机密性、split-KEM 分量来源及完整参数对应关系。其中，前两类安全性质成立，而接收方与发送方的完整参数对应关系不成立。该反例表明，即使攻击者无法获得会话密钥，仍然能够破坏发送方与接收方之间的会话参数一致性，并使接收方在错误的会话参数下完成接收。

这种攻击与传统的会话密钥泄露攻击不同。攻击者的目标不是计算双方协商的密钥，而是通过修改传输消息，使接收方接受一个未与发送方完成过程匹配的会话，从而破坏密钥交换所需的双方一致性。

3.2 攻击过程与形成原因

设合法发送方为 A，接收方为 B。根据第 1 章介绍的协议过程，发送方首先执行三组密钥封装，产生共享秘密 K_l、K_k、K_s 及相应的密文 ct_l、ct_k、ct_s。原始发送消息为：

$$m_A = (ct_l, ct_k, ct_s) \quad (8)$$

发送方根据完整消息和身份、预密钥等信息计算会话标识 sid_A，并派生会话密钥：

$$k_A = \mathit{KDF}(K_l, K_k, K_s, sid_A) \quad (9)$$

攻击者在公开网络中截获消息 m_A 后，利用以下过程构造攻击消息。

（1）截获并分解合法消息。 攻击者取得发送方公开传输的密文元组，保留其中的 split-KEM 密文 ct_s。该分量由合法发送方利用其 split-KEM 秘密材料产生，接收方能够使用相应秘密材料对其进行解封装。

（2）重新构造普通 KEM 密文。 由于接收方长期 KEM 公钥和临时 KEM 公钥均为公开材料，攻击者可以分别执行封装操作，生成新的密文 $ct_l^{'}$ 和 $ct_k^{'}$，以及对应的共享秘密 $K_l^{'}$ 和 $K_k^{'}$。这一过程不需要接收方秘密密钥。

（3）构造并投递篡改消息。 攻击者将重新生成的两个普通 KEM 密文与保留的合法 split-KEM 密文组合，得到：

$$m_B^{'} = (ct_l^{'}, ct_k^{'}, ct_s) \quad (10)$$

然后将 $m_B^{'}$ 发送给接收方 B，替代原始消息。

（4）接收方处理篡改消息。 接收方首先验证发送方的预密钥签名，然后使用本地长期 KEM、临时 KEM 及 split-KEM 秘密材料处理三个密文分量。由于三个密文分量在各自对应的解封装规则下均有效，接收方能够成功得到：

$$K_l^{'}, K_k^{'}, K_s$$

进而计算新的会话标识和密钥：

$$sid_B^{'} = \mathit{SID}(A,B,pk_A,pk_B,$$
$$prekey_A, prekey_B, m_B^{'}) \quad (11)$$

$$k_B^{'} = \mathit{KDF}(K_l^{'}, K_k^{'}, K_s, sid_B^{'}) \quad (12)$$

（5）接收方完成失配会话。 在原始接收流程中，接收方成功解封装并完成密钥派生后，即记录 $RecvDone(B,A,sid_B^{'},k_B^{'})$。此时，原发送方记录的是其根据 m_A 生成的 sid_A 和 k_A，而接收方使用的是篡改消息 $m_B^{'}$ 对应的 $sid_B^{'}$ 和 $k_B^{'}$。

在本文采用的符号构造中，会话标识绑定完整密文消息，因此上述攻击执行满足：

$$m_A \neq m_B^{'} \quad (13)$$

$$sid_A \neq sid_B^{'} \quad (14)$$

$$k_A \neq k_B^{'} \quad (15)$$

攻击最终使发送方 A 与接收方 B 得到不同的会话密钥，而 B 仍执行了接收完成操作。

该攻击成功的根本原因在于：K-Waay 原始核心接收流程缺少对完整消息及其派生会话参数的显式确认。

协议中的预密钥签名主要认证参与方预先发布的公钥材料，并不直接确认随后传输的完整密文元组。split-KEM 分量虽然提供了与合法发送方相关的密码学来源约束，但它只覆盖相应分量，不能阻止攻击者重新生成其他普通 KEM 密文。

因此，接收方能够验证部分密码分量的合法性，却未必能够确认当前派生的完整会话参数与发送方实际完成的会话一致。

需要注意的是，攻击者虽然自行生成了 $K_l^{'}$ 和 $K_k^{'}$，但无法通过公开的 ct_s 推导诚实 split-KEM 对应的 K_s，因而仍无法计算接收方派生的 $k_B^{'}$。这与 ProVerif 中接收方会话密钥机密性成立的结果一致。

综上，该攻击揭示了一个重要的安全问题：接收方密钥不泄露，并不能阻止攻击者通过篡改协议消息破坏双方的会话密钥一致性。

3.3 基于 HMAC 的接收确认加固

针对上述消息篡改攻击，本文提出在 K-Waay 核心密钥交换过程中引入基于 HMAC 的显式密钥确认机制，在保留原有 KEM、split-KEM 封装及密钥派生结构的基础上，增加接收方正式接受前的完整会话参数校验。

该机制的设计目标是：只有当接收方导出的会话参数与发送方生成确认标签时使用的参数一致，接收方才能完成当前密钥交换。

3.3.1 发送方确认标签生成

发送方按照原协议生成三组共享秘密、密文消息 m_A、会话标识 sid_A 和会话密钥 k_A 后，进一步从会话密钥导出确认密钥，并计算 HMAC 标签：

$$tag_A = \mathit{HMAC}(confirm_key(k_A), sid_A) \tag{16}$$

其中，$confirm_key$ 表示用于密钥确认的派生函数，HMAC 的输入为当前会话标识。

由于 sid_A 已包含参与方身份、公钥、预密钥材料以及完整密文消息，该标签同时关联了会话密钥和完整会话转录本。

发送方最终传输的消息由原始密文元组扩展为：

$$M_A = (m_A, tag_A) \tag{17}$$

这一修改不改变原协议的三组 KEM/split-KEM 封装方式，而是在原有密钥派生过程之后增加确认信息。

3.3.2 接收方确认检查

接收方收到 M_A 后，首先提取原始消息 m 和确认标签 tag，执行原有预密钥签名验证、密文解封装与密钥派生操作，得到本地会话标识 sid_B 和会话密钥 k_B。

随后，接收方根据本地派生结果计算：

$$tag_B = \mathit{HMAC}(confirm_key(k_B), sid_B) \tag{18}$$

并验证：

$$tag_B = tag \tag{19}$$

只有确认标签匹配，接收方才触发会话密钥接受和接收完成事件；若校验失败，则不能进入该模型中的正式接收完成路径。

这一机制使发送方与接收方的会话参数一致性成为接收完成的必要条件，而不再仅依赖各密文分量是否能够成功解封装。

3.3.3 对消息篡改攻击的阻断

考虑第 3.2 节中的攻击。攻击者重新构造 $ct_l^{'}$ 和 $ct_k^{'}$ 后，接收方处理的消息变为 $m_B^{'}$，由此派生的 $sid_B^{'}$ 和 $k_B^{'}$ 均与原发送方使用的参数不同。

因此，原发送方生成的确认标签：

$$tag_A = \mathit{HMAC}(confirm_key(k_A), sid_A) \tag{20}$$

不能作为篡改消息对应参数的有效确认标签。接收方根据自身参数计算的标签为：

$$tag_B^{'} = \mathit{HMAC}(confirm_key(k_B^{'}), sid_B^{'}) \tag{21}$$

在无密钥泄露的符号模型中，攻击者无法为这组新的会话参数构造通过检查的确认标签。

因此，原本能够触发接收完成事件的消息重组路径，在加入 HMAC 确认后被阻断。

HMAC 加固使攻击者即使能够构造可成功解封装的密文，也无法再利用该密文诱导接收方完成一个缺少匹配发送会话的执行。

需要区分的是，确认标签校验并未检查消息在同一批次中是否已经使用。因此，该机制解决的是当前核心模型中的完整参数确认问题，而不是批次重放防护问题。

3.4 HMAC 加固的形式化验证

为验证所提出机制的有效性，本文分别对原始 K-Waay 核心模型及加入 HMAC 确认的变体进行 ProVerif 分析。

两个模型均保留原有的三组密钥封装、预密钥签名检查及会话密钥派生过程。加固变体在发送方增加确认标签生成，在接收方增加标签匹配检查，并使接收完成事件受确认结果约束。原始模型在基础密文消息公开之后记录发送完成事件；HMAC 加固模型在确认消息公开之前记录发送完成事件。原始模型与加固模型分别依据基础消息发送和确认消息发送过程设置发送完成观察点，相关对应关系验证结果应结合各自事件定义理解。

对比验证主要考察会话密钥机密性、split-KEM 分量来源性质、完整参数对应关系以及正常执行可达性。

表 2 原始核心模型与 HMAC 加固模型的验证结果

安全性质	原始模型	HMAC 加固模型
发送方会话密钥机密性	成立	成立
接收方会话密钥机密性	成立	成立
split-KEM 分量来源性质	成立	成立
完整参数对应关系 P1	**不成立**	**成立**
正常执行可达性	可达	可达

注：结果对应无密钥泄露的符号配置；split-KEM 分量来源性质由独立的分量检查配置验证。

原始模型中，ProVerif 找到了 $\mathit{RecvDone}(B, A, sid, k)$ 缺少匹配 $\mathit{SendDone}(A, B, sid, k)$ 的反例，对应第 3.2 节所述的消息篡改及会话密钥失配攻击。

加入 HMAC 确认后，完整参数对应关系查询由 false 变为 true，表明所发现的接收方错误接受路径已在加固模型中得到阻断。同时，发送方和接收方会话密钥机密性、split-KEM 分量来源性质继续成立，正常密钥交换过程仍然可达。

上述结果验证了本文 HMAC 确认方案的有效性：它在无密钥泄露的符号模型中消除了所发现的完整会话参数失配接受路径，并保持了原有核心密钥交换的正常执行能力。

本文还对发送方签名秘密密钥泄露情形进行了独立验证；该配置下完整参数对应关系再次被反例否定，说明上述加固结论适用于所验证的无泄露条件，而不能直接推广至所有密钥泄露场景。

本章的研究表明，K-Waay 核心密钥交换中的密码分量合法性与双方会话密钥一致性是两个不同的安全目标。本文通过攻击建模发现了消息篡改导致的会话失配路径，并提出 HMAC 确认机制加以阻断，完成了从攻击发现到加固验证的研究过程。

然而，HMAC 确认只能保证所验证范围内接受消息与发送会话之间的参数对应关系，不能限制同一条合法确认消息被重复处理的次数。对于 K-Waay 的 BatchReceive 接口，当批次输入约束未被维持时，合法确认消息仍可能在多个槽位中被重复接受。

4 K-Waay 批内重放攻击与安全影响分析

4.1 批内重放攻击场景

第 3 章通过 HMAC 密钥确认机制阻断了消息篡改导致的会话密钥失配攻击，但有效的消息确认并不意味着消息只能被接收一次。攻击者仍可能重复投递合法确认消息，使接收方在不同槽位中多次接受同一发送会话。

针对这一问题，本文使用 Tamarin 建立 K-Waay BatchReceive 的固定两槽模型，研究批次输入约束未被维持时的接收行为。原协议要求同一批次中的输入来自不同参与方，而本文考察该条件未得到保证的情形，允许同一发送方的相同消息进入共享接收状态的两个槽位。分析发现，即使消息通过 HMAC 确认，仍可产生一个合法发送对应两次成功接收的攻击路径。

4.2 HMAC 确认后的批内重放攻击

4.2.1 攻击过程

设发送方 A 按照第 3 章的加固协议生成合法消息 m、会话标识 sid、会话密钥 k 以及确认标签：

$$tag = HMAC(confirm_key(k), sid) \tag{22}$$

发送方公开确认消息 $M = (m, tag)$，并产生一次 $ConfirmedSend(A, B, m, sid, k, tag)$ 事件。

攻击者截获 M 后，无须修改密文或伪造 HMAC，而是将同一确认消息分别投递到接收方 B 创建的批次 bid 中两个不同槽位 idx_1 和 idx_2。两个槽位共享接收状态 rst，且满足 $idx_1 \neq idx_2$。

批次封闭后，接收方依次处理两个槽位。由于两次处理使用完全相同的合法确认消息，其对应的会话参数及 HMAC 标签均能够通过验证，从而分别触发：

$$ConfirmedReceiverAccept$$
$$(B, A, bid, idx_1, rst, m, sid, k, tag) \tag{23}$$

$$ConfirmedReceiverAccept$$
$$(B, A, bid, idx_2, rst, m, sid, k, tag) \tag{24}$$

最终，接收方正常完成该批次，产生 $BatchComplete(B, bid, rst)$。

因此，攻击者仅通过重复投递一条合法消息，即可形成：

一次合法发送 → 两次确认成功 → 两次接收完成 → 批次正常结束。

该攻击不需要泄露会话密钥，也不涉及 HMAC 伪造，而是利用了批次处理过程中缺少消息级重复接受约束的问题。

4.2.2 攻击原因及验证

HMAC 标签绑定了会话标识及派生密钥，能够识别与发送会话参数不一致的消息，却不能判断同一有效消息是否已在当前批次中被处理。因此，当两个槽位使用相同的合法确认消息时，各自的密码学验证均能够成功。

本文针对该问题构造 Tamarin 攻击存在性引理 one_confirmed_send_two_accepts_exists。验证结果表明，在不发生发送方和接收方状态泄露的情况下，一个合法确认发送对应两个不同接收事件的攻击路径可达，证明过程为 17 步。

进一步验证发现，confirmed_message_accepted_at_most_once 和 injective_confirmed_receiver_accept 两项安全性质均被 15 步反例否定，而合法接收的发送来源对应关系仍然成立。

该结果表明，HMAC 修复了消息篡改引起的完整参数对应关系问题，却不能阻止同一有效消息在一个批次中被重复接受。因此，单条消息的密码学确认并不能替代批次级重复输入检查。

4.3 重复接受的安全影响

为进一步分析批内重复接受对后续会话处理的影响，本文建立独立的上层消费模型 C_install-v2，研究多个成功接收输出被分别安装时可能产生的安全后果。

4.3.1 上层消费模型

在该模型中，每个成功的 $ReceiverAccept$ 均产生一个具有独立标识 aid 的接收输出。批次成功完成后，消费组件逐项处理这些输出，并为每次安装分配新的本地句柄 h，产生：

$$InstallSession(B, h, A, sid, k) \tag{25}$$

C_install-v2 保证安装事件具有对应的接收输出来源，并约束安装发生在批次完成之后；但消费组件不依据消息、会话标识或会话密钥合并不同的成功输出。

因此，当同一消息被两个槽位接受时，模型允许得到具有相同会话参数但不同输出标识的两个结果。

4.3.2 条件化重复安装攻击

攻击者将同一合法消息重复投递至批次中的两个槽位后，接收方分别产生两个接收事件及对应输出：

$$AcceptOutputCreated$$
$$(aid_1, B, A, bid, idx_1, rst, m, sid, k) \tag{26}$$

$$AcceptOutputCreated$$
$$(aid_2, B, A, bid, idx_2, rst, m, sid, k) \tag{27}$$

其中 $aid_1 \neq aid_2$。

批次完成后，消费组件分别安装两个输出，形成：

$$InstallSession(B, h_1, A, sid, k) \tag{28}$$

$$InstallSession(B, h_2, A, sid, k) \tag{29}$$

其中 $h_1 \neq h_2$，而两个安装事件具有完全相同的发送方、会话标识和会话密钥。

这一过程表明，一次发送产生的重复接收输出，可以在逐输出独立安装的消费机制下形成两个具有相同会话参数、不同本地句柄的安装事件。

本文通过 one_send_two_accepts_two_installs_exists 验证了该攻击路径的可达性，Tamarin 给出了 28 步执行见证。同时，unique_install_within_completed_consumer 被反例否定，证明该消费模型中的同源安装唯一性无法维持。

该结果是对重复接受安全影响的进一步验证，但其安装部分依赖明确的 C_install-v2 消费假设。这里使用的是独立的原始重复接收与消费组合模型，而非第 4.2 节 HMAC-only 模型的直接扩展，因此不将两个验证结果描述成同一条联合攻击轨迹，也不据此断言实际实现必然发生会话克隆。

【图 3 预留：批内重复接受与条件化重复安装攻击。】

4.4 形式化验证结果与分析

本文分别验证原始重复接收模型、HMAC-only 模型以及 C_install-v2 消费模型。表 3 列出关键结果。

表 3 批内重放攻击与安全影响验证结果

模型	验证目标	结果	步数
原始接收	一次发送对应两次接受	可达	15
HMAC-only	合法确认消息重复接受	**可达**	**17**
HMAC-only	批内接收单射性	**不成立**	**15**
HMAC-only	接收存在匹配发送	成立	16
C_install-v2	一次发送对应两次安装	**可达**	**28**
C_install-v2	同源安装唯一性	**不成立**	**28**
C_install-v2	正常消费完成	可达	25

验证结果表明，攻击者无需伪造有效确认信息，即可在放宽输入条件的批次模型中造成重复接受。攻击执行保持正常的

槽位收集、批次封闭及接收状态消费过程，不依赖跨批次状态重用。进一步的消费模型验证表明，重复接受还可在逐输出安装条件下传播为重复符号安装。

上述研究揭示了 K-Waay 接收安全的第二个问题：单条消息的合法性验证不足以保证批次级接收唯一性，而重复接收可能进一步影响上层会话状态管理。

因此，针对本文发现的攻击，仅增加 HMAC 确认仍不充分，还需要在批次处理前实施消息级重复检测，阻止相同消息进入多个成功处理槽位。第 5 章将在此基础上设计批内原子去重机制，并验证其与 HMAC 确认的组合加固效果。

5 K-Waay 批内去重与组合加固

5.1 批内原子消息去重机制

针对第 4 章发现的批内重复接受问题，本文提出一种批内原子消息去重机制，在 BatchReceive 处理消息之前统一检查批次输入，避免同一合法消息在多个槽位中被重复接受。

设同一批次包含消息 m_1 和 m_2，其中 $m = (ct_l, ct_k, ct_s)$ 为完整密文元组。接收方首先判断：

$$m_1 = m_2 \tag{30}$$

若相等，则判定存在重复消息，直接拒绝整个批次，不执行后续槽位处理；若不相等，则允许进入原有的密文解封装和密钥派生流程。

为保证去重操作的原子性，本文将检查设置在批次封闭之后、任何槽位处理之前，并使用线性状态约束同一批次只能执行一次去重决策。因此，重复消息不会产生成功接收事件，也不会形成可供后续消费的接收输出。

该机制不需要修改原有密码学原语，仅在批次处理入口增加消息唯一性检查，即可阻断同一消息的批内重复接受。

5.2 原子去重机制的验证

本文使用 Tamarin 对加入原子去重机制的接收过程进行验证。

结果表明，原先可达的“一次发送、两次接收”攻击在修复后不再可达，批内相同消息至多接受一次及接收发生单射性均得到验证。

对于重复输入，模型能够正常触发批次拒绝，且不会产生成功接收事件。在逐输出消费的条件下，重复接收导致的双重安装路径也被阻断。

同时，两个不同且有效的消息仍能够完成正常批次处理，说明该机制能够在阻止重复接受的同时保留正常接收能力。

5.3 HMAC 与原子去重的组合加固

为同时解决消息篡改导致的会话密钥失配和批内重复接受问题，本文将第 3 章的 HMAC 确认机制与原子去重机制结合，提出组合加固方案。

改进后的接收过程分为三个阶段。

首先，执行批内去重。 接收方收集完整批次后，提取各输入中的基础消息 m，并检查是否存在重复。若存在重复消息，则直接拒绝批次。

其次，执行密钥确认。 对于通过去重检查的消息，接收方执行原有密文解封装与密钥派生，并验证：

$$tag_i = \mathit{HMAC}(confirm_key(k_i), sid_i) \tag{31}$$

只有确认标签正确，才能接受相应消息。

最后，完成批次处理。 所有槽位均成功处理后，接收方完成批次并输出对应结果。

组合方案使用基础密文消息 m 作为去重对象，而不是完整的 (m, tag)。因此，即使同一消息携带不同确认标签，也会被识别为重复输入，不能绕过去重检查。

通过上述改进，原子去重负责阻止重复接受，HMAC 负责确认发送与接收双方的会话参数一致性，从而实现对两类安全问题的联合防护。

【图 4：K-Waay 组合加固后的接收处理流程，待插入。】

5.4 组合加固的安全性验证

本文进一步使用 Tamarin 验证组合加固后的接收过程，并与修复前的结果进行对比，如表 4 所示。

表 4 接收端加固前后的安全性质对照

验证目标	修复前	修复后
相同消息重复接受	可达	不可达
批内接收单射性	不成立	成立
HMAC 有效消息重复接受	可达	不可达
同一消息携带不同标签	—	重复拒绝
条件化重复安装	可达	不可达
不同有效消息正常接收	可达	可达

注：重复安装结果基于第 4 章定义的逐输出消费条件。

验证结果表明，组合加固能够阻断合法消息的批内重复接受，保证相同基础消息不会在同一批次中产生多个成功接收事件。对于确认标签不正确的消息，接收方也无法完成相应的确认接收。

此外，正常双消息接收和后续输出消费均保持可达，表明组合方案在满足安全检查要求的同时保留了正常批量接收功能。

第 3 章的 ProVerif 验证表明，HMAC 密钥确认能够在无密钥泄露的核心模型中恢复完整会话参数对应关系。本节 Tamarin 验证进一步表明，在固定两槽批次模型中，原子去重与 HMAC 组合能够阻断相同消息的重复接受，并在指定消费条件下排除相应的重复安装路径，同时保留不同有效消息正常接收与消费的可达性。

6 结束语

本文针对 K-Waay 密钥交换协议的接收端安全问题开展形式化分析。首先，通过 ProVerif 发现攻击者能够重组 KEM 密文，使接收方接受与合法发送会话不匹配的会话参数，即使攻击者无法获得相应的会话密钥。针对该问题，本文提出基于 HMAC 的密钥确认机制，并验证其能够在无密钥泄露条件下恢复完整会话参数对应关系。

进一步地，通过 Tamarin 发现，当 BatchReceive 的参与方互异前置条件未得到维持时，即使引入 HMAC 确认，合法消息仍可能在同一批次中被重复接受，并在特定上层消费条件下产生重复安装。为此，本文提出批内原子消息去重机制，并将其与 HMAC 确认相结合。形式化验证表明，组合方案能够阻断相应的消息篡改与批内重复接受攻击，同时保持正常批次处理与输出消费的可达性，为 K-Waay 接收端安全加固提供了可验证的改进方案。

参考文献

[1] CREMERS C, JACOMME C, NASKA A. Formal analysis of session-handling in secure messaging: lifting security from sessions to conversations[C]//32nd USENIX Security Symposium (USENIX Security 23). Anaheim, CA: USENIX Association, 2023: 1235-1252.

[2] COLLINS D, HUGUENIN-DUMITTAN L, NGUYEN N K, et al. K-Waay: fast and deniable post-quantum X3DH without ring signatures[C]//33rd USENIX Security Symposium (USENIX Security 24). Philadelphia, PA: USENIX Association, 2024: 433-450.

[3] COLLINS D, HUGUENIN-DUMITTAN L, NGUYEN N K, et al. K-Waay: fast and deniable post-quantum X3DH without ring signatures[EB/OL]. Cryptology ePrint Archive, Paper 2024/120, 2024[2026-10-08]. https://eprint.iacr.org/2024/120.

[4] COHN-GORDON K, CREMERS C, DOWLING B, et al. A formal security analysis of the Signal messaging protocol[J]. Journal of Cryptology, 2020, 33(4): 1914-1983. DOI:10.1007/s00145-020-09360-1.
[5] BHARGAVAN K, JACOMME C, KIEFER F, et al. Formal verification of the PQXDH post-quantum key agreement protocol for end-to-end secure messaging[C]//33rd USENIX Security Symposium (USENIX Security 24). Philadelphia, PA: USENIX Association, 2024: 469-486.
[6] MEIER S, SCHMIDT B, CREMERS C, et al. The TAMARIN prover for the symbolic analysis of security protocols[C]//Computer Aided Verification: CAV 2013. Berlin, Heidelberg: Springer, 2013: 696-701. DOI:10.1007/978-3-642-39799-8_48.
[7] LUPETTI S, DILLEMA F W, STABELL-KULO T. Names in cryptographic protocols[C]//Proceedings of the 4th International Workshop on Security in Information Systems. Paphos, Cyprus: SciTePress, 2006: 185-194. DOI:10.5220/0002484701850194.
[8] LOWE G. A hierarchy of authentication specifications[C]//Proceedings of the 10th Computer Security Foundations Workshop. Rockport, MA: IEEE Computer Society Press, 1997: 31-43. DOI:10.1109/CSFW.1997.596782.